RESEARCH NOTE / CHECKED LEAN CANDIDATES

Further Lean proof simplifications

DiophantineConsistency, beyond the existing proposals

26 October 2026

Main finding. The strongest immediate reduction is a shared Pell ratio estimate whose error allowance is a parameter. Two long proofs become one theorem and two short wrappers. A sharper native-rank inequality removes a second mathematical duplication. Integer induction centered on the known point substantially simplifies a Wang interval argument. A bounded first-failure certificate isolates repeated minimal-index reasoning. A shared-edge representation of Wang rows is a promising structural follow-up, with a checked generic kernel and an explicit remaining integration obligation.

Deliverable. This note accompanies Lean examples, individual and combined patches, complete candidate source files, compiler and axiom receipts, an exclusion inventory, and portable replay scripts. The supplied patches preserve the existing public theorem statements. The recommended interval patch also preserves its exact former axiom set.

Scope. Source calculations and compiler checks use ProveIt commit

`4b3dd4efb10f1af27143a61423803221319687 26`.

The exclusion corpus was refreshed at commit

`23f631ea07fff8632eb8cd92286f7621bbe051c0`.

The seven patched source files and the shared-edge proposal's target were unchanged between these commits. The later incoming proposals are reviewed separately in the accompanying freshness audit.

How to read the evidence. Complete candidate-module compilation, an isolated exact theorem, and a generic design kernel are different levels of evidence. They are labeled separately throughout. These checks reused dependency artifacts; they are not a clean rebuild of the entire repository. Physical line counts include helpers and wrappers where specified. They do not imply faster elaboration or a total saving relative to every possible implementation of the earlier proposals.

This is a scoped non-repetition study of the supplied repository material. It makes no claim that the underlying mathematics or library APIs are new to the wider Lean literature.

Contents

1 Selection rule and evidence **3**
1.1 Recommendation table . . . 4
1.2 Baseline and counterfactual accounting . . . 4

2 One Pell estimate with a parameterized error allowance **4**
2.1 Where the duplication occurs . . . 4
2.2 Why the generalization works . . . 5
2.3 Proposed API and migration . . . 5
2.4 Measured outcome and evidence . . . 6

3 A sharper common lower envelope for the native rank **6**
3.1 Extract the stronger mathematical fact . . . 6
3.2 Two clients of one nonlinear derivation . . . 6
3.3 What is sharp, and what is established . . . 7

4 Use integer induction around the known point **7**
4.1 Choose the induction domain that matches the theorem . . . 7
4.2 A shorter alternative and its axiom cost . . . 8

5 Return the first failure together with its valid prefix **8**
5.1 The repeated administrative argument . . . 8
5.2 Boundary cases and source impact . . . 9

6 Make Wang horizontal matching structural **9**
6.1 A different proof view of the same row . . . 9
6.2 The machine step has one nonzero bridge edge . . . 10
6.3 Preserve the existing unconditional statements . . . 10
6.4 Checked core and remaining experiment . . . 11

7 Small additional cleanup: absolute-value trace bounds **11**

8 Validation, adoption and reproducibility **11**
8.1 What was checked . . . 11
8.2 Suggested adoption order . . . 12
8.3 Replaying the bundle . . . 12

9 Questions worth pursuing next **13**

A Artifact map **13**

B Primary sources and exact targets **14**

1 Selection rule and evidence

The request is stronger than finding code that has not yet been edited: ideas already described in the research packets or incoming material must be treated as implemented. Accordingly, adoption status was ignored when deciding novelty. Deferred directions, uncompiled sketches, supplementary source notes and further-research questions all count as prior work.

The initial inventory covers all six placed packets, a–f, under `research/lean-proof-simplification`, their README and adoption record, and all five incoming ZIPs then present. It records 323 placed files and 400 nondirectory archive members. Of those archive members, 251 text, source, patch and metadata files were extracted for inspection; the manuscripts were read from their TeX sources. PDFs and generated files were hashed without treating old compiler outputs as evidence for the present candidates. The incoming README was searched throughout, and its relevant intake summaries were read. The initial exclusion ledger has 83 mechanisms, each linked to its supporting prior material. The supplementary freshness audit covers the four ZIPs added while this work was in progress. The final review therefore covers fifteen packets: six placed packets and nine incoming ZIPs, with 2,216 incoming nondirectory members and 2,048 extracted text/code members. Its consolidated ledger has 98 excluded mechanisms. It caused a small-square refactoring of `firstUnit_fundamental` to be withdrawn: the new `additional_lean_simplifications` packet already replaces that target by a uniform small-ordinate criterion. The withdrawn study is not included among the delivered proposals or patches.

The full audit is in `audit/novelty_audit.md`, with machine-readable inventories and exclusion records beside it. The method establishes non-repetition within the inspected material at the stated commits. It does not recursively claim coverage of every retired manuscript linked outside the two requested directories.

Three distinctions prevent misleading novelty claims here:

- Replacing artificial existential witnesses by `set` in the native ratio proof was already proposed. Only sharing the scale-dependent inequality is credited as new, and that older cleanup is explicitly subtracted in the ratio comparison.
- A source-facts interface and native-rank cast conversions were already proposed. The new rank contribution is the stronger factor $2q-1$, derived from the positive transport gap. Merely placing old facts in a record, or shortening casts, contributes no new idea.
- Generic semantic locality, scanner iteration, finite-table dispatch and controller invariants were already proposed. The Wang representation below instead makes a particular geometric matching condition structural by sharing one edge function between adjacent tile records.

1.1 Recommendation table

Proposal	Concrete payoff	Validation
Parameterized ratio estimate	70 physical lines removed across two files; 66 after the explicit earlier `set` cleanup	Both complete changed modules; public wrappers and downstream axiom audit
Sharper native-rank envelope	$R \geq (2q-1)(q^2-1)$, hence $R \geq 40$ and $R > 3q$	Both complete changed modules; generic 40-bound; combined arithmetic check
Integer induction at the seed	26-line proof body becomes 14 lines with the old axiom footprint	Complete module and existing focused client
Bounded first failure	Two minimality blocks share one prefix contract; 2 net module lines saved	Complete module and existing focused client
Shared-edge Wang row	Horizontal matching becomes one index identity	Generic kernel checked; actual row equality and membership integration pending

A separate five-line trace cleanup is included as a maintenance bonus. It is not counted among the five main proposals.

1.2 Baseline and counterfactual accounting

The patches target a real, reproducible source snapshot. There is no single concrete tree representing all mutually optional sketches in the prior packets. Thus the conceptual baseline assumes all those ideas, while the patch and line-count baseline is the pinned code. When adopting these suggestions after the older work, place the new facts into the already established interfaces and retain only the still-needed wrappers.

This matters especially for the rank patch. The two-file total changes from 769 to 765 lines, but the four physical lines saved include a cast simplification already within the earlier proposals' scope. They should not be advertised as four novel lines saved. The new inequality is useful even if the fully refactored baseline makes its net source cost neutral. No aggregate number in the bundle is presented as a guaranteed incremental saving after every previous idea.

2 One Pell estimate with a parameterized error allowance

2.1 Where the duplication occurs

`NativeHalfBinomial.lean` contains `upper_estimate_scaled`. Its existing import, `PellRatio90.lean`, contains `Jones1980.Ratio90.upper_estimate'`. Both declarations occupy 91 lines in the pinned source. They repeat positivity, Pell growth, perturbation and denominator-cancellation arguments. Their decisive difference is how much room is allowed for the ratio and its additive error.

Write

$$\xi = \frac{(U+1)^{2r}}{U^r}.$$

The upstream proof assumes $\xi < Y + 1$ and proves $c/k < \xi + 1/2$. The native version assumes $\xi < 2(Y+1)$ and proves $c/k < \xi + 1$. The shared structure is the implication

$$\kappa > 0, \qquad \xi < \kappa(Y+1) \quad \Longrightarrow \quad \frac{c}{k} < \xi + \frac{\kappa}{2}. \tag{1}$$

The other numerical and Pell hypotheses remain those of the existing arguments. In particular, $U, Y, r \geq 1$, the sequence identities for c, k, $a = Y(U+1)$, $J = 2r+1$, $72r < U+1$, and $16 \leq Y(U+1)$ are retained. Equation (1) is a description of the new conclusion under those hypotheses, not an unconditional numerical implication.

2.2 Why the generalization works

Put $\varepsilon = 9/[Y(U+1)]$ and $\delta = 2r\varepsilon$. The existing growth argument supplies

$$c \leq k\xi(1+\delta), \qquad k > 0.$$

Its small-perturbation estimate gives

$$(Y+1)\delta < \frac{1}{2}.$$

For intuition, this last left side is

$$\frac{18r(Y+1)}{Y(U+1)} \leq \frac{36r}{U+1} < \frac{1}{2},$$

using $Y \geq 1$ and $72r < U+1$. The formal argument retains the original positivity obligations and clears only positive denominators.

Since $\delta \geq 0$,

$$\xi\delta \leq \kappa(Y+1)\delta < \frac{\kappa}{2}.$$

Combining the two displays and dividing by positive k proves (1). Thus the numerical scale belongs in the statement of the common theorem, while the growth and perturbation machinery is shared unchanged.

2.3 Proposed API and migration

The complete patch introduces `upper_estimate_of_ratio_envelope` in the upstream ratio module. The original upstream theorem specializes it at $\kappa = 1$; the native theorem specializes it at $\kappa = 2$. A compact view of the wrappers is:

```
-- Upstream wrapper: original hypotheses and conclusion retained.
simpa only [one_mul] using upper_estimate_of_ratio_envelope
  hU hY hr hJ hA ha hP hc hk (κ := 1) (by norm_num)
  (by simpa only [one_mul] using hξY') h72 h16

-- Native wrapper: original hypotheses and conclusion retained.
simpa only [show (2 : ℝ) / 2 = 1 by norm_num] using
  upper_estimate_of_ratio_envelope hU hY hr hJ hA ha hP hc hk
    (κ := 2) (by norm_num) hξY' h72 h16
```

The upstream wrapper keeps its existing `hYr : U ^ r ≤ Y` argument even though the generalized core does not use it. Removing a public hypothesis merely because it becomes unused would create an unnecessary interface change. The existing downstream doubled-coordinate calculation in the native half-binomial theorem remains separate. The new scale parameter does not justify merging that step or changing its constant.

This placement uses an import edge already present in the native module. If the project prefers to avoid modifying the upstream file, an intermediate lemma module can host the generalization, but its complete source cost and dependency direction should be assessed before choosing that arrangement. The supplied candidate takes the simpler existing dependency direction.

2.4 Measured outcome and evidence

The upstream file grows from 472 to 484 lines; the native file shrinks from 244 to 162. The combined total is therefore 716 to 646, including the shared theorem and both wrappers. The declarations change from $91 + 91$ lines to $92 + 8 + 9$ lines.

The earlier incoming notes already recommend `set` in the analytic proof. Modeling that exact cleanup on both old copies reduces their combined starting total by four lines, to 712. Relative to this explicit counterfactual, the new factoring removes **66 lines**. This is the relevant compression claim; the separate four-line cleanup receives no new credit.

Both complete changed modules compile. Axiom reports cover the new theorem, both retained wrappers, `Jones1980.Ratio90.upper_estimate` and `source_half_binomial`; their lists contain only `propext`, `Classical.choice` and `Quot.sound`. The standalone `lean/RatioFactor.lean` checks the same mathematical mechanism under a separate namespace. Its helper name differs from the production candidate to avoid accidental ambiguity. Compiler timings in the receipts are diagnostic observations, not benchmarks.

3 A sharper common lower envelope for the native rank

3.1 Extract the stronger mathematical fact

Two existing proofs reconstruct the same positive transport gap and leading product:

- `source_native_rank_ge_twentyfour` in `NativePellBootstrap.lean`;
- `source_rank_gt_three_q` in `NativePellScaleBounds.lean`.

Earlier proposals already cover exposing these source facts. The additional observation is that the gap proves a stronger factor than either current proof uses.

Use the source notation

$$q = 1 + x_0\beta, \quad F = x_1, \quad \alpha = x_2, \quad C = q - F - \alpha > 0.$$

All these inequalities are interpreted over integers, and F, α are positive. Consequently,

$$q \geq 3, \qquad q - F = C + \alpha \geq 2.$$

The leading factor

$$G = (q-1)(q-F) + \alpha$$

therefore satisfies

$$G \geq 2(q-1) + 1 = 2q - 1. \tag{2}$$

The literal source rows identify the native rank with

$$R = G(q^2 - 1) + (M_C + M_F q)x_0.$$

Under the original nonnegative mask-port assumptions, the last summand is nonnegative. Equation (2) then gives the shared theorem `source_native_rank_envelope` in `NativePellBootstrap`:

$$q \geq 3 \quad \text{and} \quad (2q-1)(q^2-1) \leq R. \tag{3}$$

3.2 Two clients of one nonlinear derivation

For $q \geq 3$, we have $q^2 - 1 \geq 8$ and $2q - 1 \geq 5$. Hence (3) implies

$$R \geq 40.$$

It also implies $R > 3q$: indeed,

$$(2q-1)(q^2-1) \geq 8(2q-1) = 16q - 8 > 3q.$$

The old 24-bound is retained as a wrapper so existing callers remain unchanged. The scale-bound theorem also keeps its exact natural-number conclusion, converting the already obtained integer inequality at the final interface.

The crucial Lean fragment from the shared proof is:

```
have hG : 2 * q - 1 ≤ (q - 1) * (q - x 1) + x 2 := by
  nlinarith [mul_nonneg (show 0 ≤ q - 1 by omega)
    (show 0 ≤ q - x 1 - 2 by omega)]
have hGL := mul_le_mul_of_nonneg_right hG
  (show 0 ≤ q ^ 2 - 1 by nlinarith)
```

The explicit nonnegative product supplies the nonlinear fact; the solver then combines it with the source identity. This keeps the mathematical reason for the bound visible.

3.3 What is sharp, and what is established

The envelope is attained by the abstract arithmetic values $q = 3$, $F = 1$, $\alpha = 1$, $C = 1$ and zero mask term, producing $G = 5$ and $R = 40$. This establishes sharpness for those reduced arithmetic assumptions. It does *not* establish that these values extend to a solution of all the original source equations and port constraints. The bundle deliberately makes no such attainability claim.

The complete Bootstrap and ScaleBounds candidates compile, including a final check of ScaleBounds against the freshly built Bootstrap object after the compatible patches were composed. `lean/RankAxioms.lean` proves the generic implication to 40 and checks the old public endpoints. No theorem assumes consistency, an auxiliary run, or a new source certificate beyond its original hypotheses.

After the prior source-facts proposal is assumed applied, place (3) beside those facts and make both clients consume it. The contribution is the improved inequality and shared nonlinear derivation. The physical two-file reduction of four lines is reported only as pinned-source accounting; the cast cleanup in that patch belongs to previously proposed work.

4 Use integer induction around the known point

4.1 Choose the induction domain that matches the theorem

The theorem `interval_propagate` in `WangTiles942ResetRows.lean` states that a predicate known at x_0 propagates in both directions throughout the open interval (l, r). The existing proof separately establishes $P(x_0 + n)$ and $P(x_0 - n)$ by natural induction, splits on the order of an arbitrary integer x, and reconstructs x with naturalized differences.

The proof obligation is already an integer induction. Translate to $z = x - x_0$ and use the motive

$$l < x_0 + z \longrightarrow x_0 + z < r \longrightarrow P(x_0 + z).$$

The zero case is the given point. In the successor and predecessor cases, integer arithmetic transports interval membership to the preceding step. There is no need to expose two natural-difference conversions at the call site.

The recommended complete replacement is:

```
have shifted (z : ℤ) : l < x₀ + z → x₀ + z < r → P (x₀ + z) := by
  induction z using Int.induction_on with
  | zero => exact fun _ _ =>
      (show P (x₀ + 0) from (by simpa only [add_zero] using h₀))
  | succ n ih =>
      intro hl hr
      simpa only [add_assoc] using
        hnext (x₀ + (n : ℤ)) (by omega) (by omega)
          (ih (by omega) (by omega))
  | pred n ih =>
      intro hl hr
      simpa only [add_sub_assoc] using
        hprev (x₀ + -(n : ℤ)) (by omega) (by omega)
          (ih (by omega) (by omega))
intro x hl hr
have he : x₀ + (x - x₀) = x := by omega
simpa only [he] using shifted (x-x₀) (by omega) (by omega)
```

This display is wrapped for readability; the measured candidate body occupies 14 physical source lines. The original body has 26.

4.2 A shorter alternative and its axiom cost

Mathlib also supplies `Int.inductionOn'` with an arbitrary base. It gives a nine-line proof body by choosing x_0 directly. Both versions preserve the exact original theorem statement and all open-interval inequalities.

There is a measurable logical difference at this version of the library. The old theorem and the translated `Int.induction_on` proof report exactly

```
[propext, Quot.sound].
```

The shorter `Int.inductionOn'` proof reports

```
[propext, Classical.choice, Quot.sound].
```

The added axiom is a standard Lean axiom, but preserving the existing footprint is a useful project constraint. The combined patch therefore uses the translated 14-line proof. The optional nine-line version is shipped independently and must not be applied on top of the recommended alternative.

Including the explicit import, the recommended complete module changes from 162 to 151 lines; the optional version would give 146. The complete recommended module and its original focused check module compile. The optional version is checked as the exact isolated theorem, not as a separate complete module build.

This remains useful after the earlier scanner refactorings: it changes the induction used for a geometric interval argument, independent of any encoded recurrence or scanner state.

5 Return the first failure together with its valid prefix

5.1 The repeated administrative argument

In `WangTiles942InitialStrip.lean`, both `left_wall_exists` and `right_wall_exists` choose a least index where an interior condition fails. Each proof establishes an existential witness, invokes `Nat.find`, gets the endpoint bound and reconstructs the valid prefix by contradiction.

The geometric arguments are asymmetric. The left proof starts at index 1, samples $T(-j)$ and uses color 2; the right proof starts at index 2, samples $T(j)$ and uses color 4. Those distinctions should remain in their respective proofs. The shared fact is only the minimality contract:

```
theorem first_failure_between (P : ℕ → Prop) (lo hi : ℕ)
    (hlo : lo ≤ hi) (hfail : ¬ P hi) :
    ∃ k, (lo ≤ k ∧ ¬ P k) ∧ k ≤ hi ∧
      ∀ j, lo ≤ j → j < k → P j := by
  classical
  have hex : ∃ k, lo ≤ k ∧ ¬ P k := ⟨hi, hlo, hfail⟩
  refine ⟨Nat.find hex, Nat.find_spec hex,
    Nat.find_min' hex ⟨hlo, hfail⟩, ?_⟩
  intro j hj hlt
  exact Classical.byContradiction
    (fun hn => Nat.find_min hex hlt ⟨hj, hn⟩)
```

This statement returns exactly the data the caller needs: failure at k, an upper bound by a known failing endpoint, and the predicate at every admissible earlier point. It requires neither a monotone predicate nor a decidable predicate argument. Classical reasoning is explicit in the helper.

5.2 Boundary cases and source impact

If $lo = hi$, the selected point is that endpoint and the required prefix is empty. The case $lo = hi = 0$ is therefore valid. A tempting helper that omits $lo \le j$ from the prefix statement would incorrectly demand the predicate before the allowed starting range. Keeping the lower bound explicit makes the two source applications straightforward.

The two ten-line administrative blocks become two two-line applications. The helper costs 12 lines including its documentation, and the import and spacing also count. The whole module changes from 148 to 146 lines: **two net lines**. The benefit is a common off-by-one contract and a clearer separation between least-index selection and wall classification. This is a small maintainability change, not a large compression claim.

The complete patch retains the original local conjunction shapes, so the geometric proof tails are textually unchanged. The modified module and its existing focused client compile. The standalone example proves the general helper without importing the tile language.

6 Make Wang horizontal matching structural

6.1 A different proof view of the same row

The current `WangTiles942StepConstruction.lean` defines `stepTile` by selecting a transition tile at the old head, a receiving tile at the new head and a passing tile elsewhere. Several subsequent field proofs repeat that partition. Horizontal matching is then shown by reconciling independently written west- and east-edge tests.

Instead, give the row one edge function $e : \mathbb{Z} \to \mathbb{N}$, alongside north and south functions n, s. Form the tile at x by

$$\mathrm{row}(x) = \langle e(x-1), n(x), e(x), s(x) \rangle, \tag{4}$$

using the repository's field order: west, north, east, south.

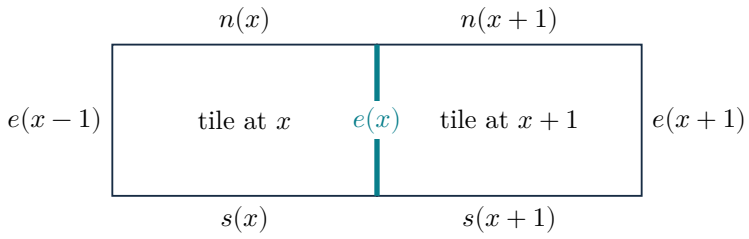


Figure 1: The shared physical edge has one source value. Matching follows from indexing.

The horizontal theorem becomes the identity $e((x+1)-1) = e(x)$:

```
theorem rowFromEdges_horizontal (north south edge : ℤ → ℕ) (x : ℤ) :
    (rowFromEdges north south edge (x + 1)).west =
      (rowFromEdges north south edge x).east := by
  exact congrArg edge (by omega)
```

North and south projection theorems are reflexive. This constructor alone says nothing about membership in the finite allowed tile language; that is a separate obligation.

6.2 The machine step has one nonzero bridge edge

Let p be the old head and d the actual action direction, with true meaning right. The new head is $p+1$ for a right move and $p-1$ for a left move. Define

$$b = \begin{cases} p & \text{right}, \\ p-1 & \text{left}, \end{cases} \qquad e(x) = \begin{cases} \rho & x = b, \\ 0 & x \neq b, \end{cases}$$

where ρ is the actual `rail` value for the direction and target, including the halt target. Equivalently, $b = \min(p, \text{newHead})$.

The checked generic kernel establishes the expected edge pairs:

Position	Move right: (west,east)	Move left: (west,east)
Old head	$(0, \rho)$	$(\rho, 0)$
New head	$(\rho, 0)$	$(0, \rho)$
Every other position	$(0, 0)$	$(0, 0)$

This is the real source of the current west/east case distinctions, expressed once in edge coordinates.

6.3 Preserve the existing unconditional statements

The actual source deliberately gives `stepTile_allowed` and `stepTile_horizontal` without a premise identifying `cfg.state` with the supplied state `q`. Only the north/south interpretation lemmas have that premise. A naive edge view using $n = \text{color}(cfg)$ would lose this distinction.

Use

```
let liveCfg := { cfg with state := some q }
```

in the proof view. Its tape and head are unchanged, and its operating state is the supplied one. Take north colors from `liveCfg` and south colors from the actual machine step of `liveCfg`. The action and

bridge are computed from the same tape bit and state as the existing construction. This makes the proposed row equality unconditional. The public north/south statements then specialize using their original premise `hq`.

The concrete integration should keep the executable `stepTile` definition and prove one equality with the edge-built view. The proof of that equality still has the three meaningful cases: old head, new head and elsewhere. It uses the existing machine color facts and the bridge-edge lemmas. Afterward, the field and matching theorems can derive from that equality. Allowedness must be preserved through the existing transition, receive and pass constructors and the real direction-specific target sets.

6.4 Checked core and remaining experiment

`lean/SharedEdgeRow.lean` uses a separate four-field record with the same field order as the repository tile. It checks the generic matching theorem, bridge/minimum identity and all three edge-pair cases. It intentionally does not masquerade as a complete repository refactor.

The equality with the actual `stepTile` and its concrete `Allowed` integration are **not implemented**. Until those are written, no net line saving is claimed. The experiment succeeds only if the cost of the equality and adapters is outweighed by simpler field and matching clients. The representation could still be useful for later row constructors even if the first migration is source-neutral.

This proposal neither assumes left/right symmetry of the whole tile language nor alters seam or initializer conventions. Its symmetry is limited to locating the bridge of one actual transition. That narrow scope avoids a false generalization over asymmetric seam colors.

7 Small additional cleanup: absolute-value trace bounds

In `WangMachineTraceBounds.lean`, `run_read_of_abs_ge` converts a bound on $|x|$ to the disjunction already accepted by `run_read_outside`. The existing proof splits on the sign of x. The library equivalence supplies the conversion directly:

```
exact run_read_outside M t x (le_abs'.mp hx)
```

The unchanged theorem checks in the complete file, and the downstream `halted_trace_box` axiom report remains standard-only. The six-line body becomes one line, removing five lines from the module. This is a useful local cleanup with a precise check, not a new abstract framework or evidence of a speedup.

8 Validation, adoption and reproducibility

8.1 What was checked

The compiler is Lean 4.32.0, release commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`. The repository pins mathlib tag `v4.32.0`; its manifest resolves to `81a5d257c8e410db227a6665ed08f64fea08e997`. Exact commands, candidate hashes and exit statuses accompany the bundle.

All seven proposed changed source modules were compiled in their complete form. Bootstrap was additionally compiled in the complete integrated candidate set, followed by ScaleBounds against that freshly built object. The ratio and native half-binomial modules were checked as a compatible pair. The Wang changes were checked with their original ResetRowsChecks and InitialStripChecks clients. Six unchanged small Wang support modules were compiled to supply those clients. The trace baseline and candidate were both compiled, with its BusyBeaver Core dependency compiled from pinned source.

These checks reused installed mathlib and existing project dependency objects. The broader Wang cache records origin commit `9bd29079d5e1c6cc1c0bcaee6cb57d59b835cba9`; the cached Language, Core, Data and DecimalMachine215 source files were individually verified byte-identical to the pinned snapshot. External BusyBeaver dependencies for that Wang closure were reused. The validation supports complete candidate-source elaboration and focused-client compatibility. It does not establish a fresh build of every dependency or all repository endpoints.

Axiom reports contain no `sorryAx`. The usual three axioms appear in the arithmetic and classical least-index proofs. The recommended interval theorem retains its smaller set. The checked generic shared-edge construction introduces no axiom declarations. The optional centered interval proof's extra `Classical.choice` is disclosed in Section 4.

The bundle preserves a resolved initial import probe that failed because a geometry object was missing. That dependency was subsequently compiled and the focused client passed. This environmental probe is kept distinct from the successful final check results. Likewise, no isolated example is represented as proof of the unimplemented actual shared-edge adapter.

8.2 Suggested adoption order

1. Adopt the parameterized ratio theorem and its two wrappers. It gives the clearest immediate reduction and retains the existing dependency direction.
2. Add the stronger rank envelope to the already planned source-facts layer. Derive the old 24-bound and the scale-bound result from it; expose a 40-bound if a client needs that statement.
3. Apply the translated integer-induction proof. Choose the optional centered eliminator only if the slightly larger standard-axiom footprint is acceptable.
4. Adopt the least-failure helper when its prefix contract improves local readability. Its modest net source reduction does not by itself justify a large shared utility framework.
5. Trial the actual shared-edge adapter in a separate change. Measure its full definition, equality, allowedness and client cost before replacing the current proof organization.

The combined patch includes the concrete checked changes and the trace cleanup. It excludes the generic shared-edge design and excludes the optional centered interval alternative. Individual patches allow each proposal to be assessed independently. Public theorem signatures were compared with the pinned source; byte-identical header comparisons are recorded separately.

8.3 Replaying the bundle

Read `README.md` and run the integrity checker before using the patches. A clean checkout at the source commit makes the line counts and source hashes reproducible. The replay script reports its plan by default; applying the patch is an explicit option. A normal dependency-resolving replay uses the repository's own Lake configuration and pinned toolchain. The provided scripts do not require the local cache paths appearing in historical receipts.

The portable scripts are syntax-checked and their integrity operations are exercised. A fresh network-dependent checkout and full clean dependency rebuild were not run as part of this report. On a later branch, apply the mathematical changes through the interfaces already adopted there, then recompile the relevant clients. The recorded source hashes make any drift visible.

Compile this report with XeLaTeX twice:

```
xelatex -interaction=nonstopmode -halt-on-error report.tex
xelatex -interaction=nonstopmode -halt-on-error report.tex
```

No shell escape, generated image or external bibliography is needed. The report uses standard TeX packages and the Latin Modern and DejaVu fonts.

9 Questions worth pursuing next

The most valuable remaining experiment is the actual shared-edge equality. Does one case analysis plus field projection reduce the total size of step-row proofs, including allowedness and the state-identification adapters? The generic kernel isolates exactly what is already known and what must still be tied to the real machine.

For the rank envelope, can the full source equations force a stronger bound than 40? The present reduced arithmetic example attains 40, but it says nothing about realizability in the full source system. Any improvement must use additional genuine source constraints rather than silently assume them. Even if the constant stays 40, identifying which later estimates need only the envelope may shorten their hypotheses.

For the ratio theorem, a future caller with a different envelope $\kappa(Y + 1)$ can reuse the same proof. Generalizing further should be driven by an actual client: the current $\kappa/2$ theorem already removes the observed duplication, and a more abstract perturbation interface might cost more than it saves. The exact wrappers provide a stable place to compare those choices.

For the least-failure helper, reuse should follow a matching mathematical contract. Another prefix-selection proof would justify moving the helper to a shared module. Its current local placement avoids importing machinery that the existing clients do not need.

A Artifact map

Path	Purpose
`report.tex`, `report.pdf`	Complete narrative, mathematics, evidence and limitations
`patches/combined.patch`	Compatible concrete changes against the source pin
`patches/individual/`	Independently reviewable recommended patches
`patches/optional/`	Shorter centered interval alternative, separately checked
`candidate-source/`	Complete composed candidate source files; repository-relative paths
`lean/`	Small checked examples and public axiom audits
`evidence/`	Compiler logs, source hashes, metrics, integration and validation receipts
`audit/`	Non-repetition inventory, exclusion judgments and freshness review
`scripts/`	Portable integrity and replay utilities
`MANIFEST.sha256`	Checksums of delivered files, excluding the checksum file itself

Compiled dependencies and old incoming ZIPs are omitted from the deliverable. Their paths and hashes remain in the inventories. This keeps the bundle small and avoids presenting historical object files as a reproducible source environment.

B Primary sources and exact targets

All ProveIt links below are pinned to the source commit given on the title page. The freshness record supplies the later comparison and archive hashes; the final incoming exclusion snapshot is pinned separately.

S1. NativeHalfBinomial, `upper_estimate_scaled`, and PellRatio90, `upper_estimate'`. These are the two analytic proof sources.

S2. NativePellBootstrap, especially `source_transport_positive`, `source_rank_eq_target` and `source_native_rank_ge_twentyfour`.

S3. NativePellScaleBounds, `source_rank_gt_three_q` and its natural/integer interface.

S4. WangTiles942ResetRows, `interval_propagate`.

S5. WangTiles942InitialStrip, left and right wall existence.

S6. WangTiles942StepConstruction, row construction, field interpretation and horizontal matching.

S7. WangMachineTraceBounds, `run_read_of_abs_ge` and `halted_trace_box`.

S8. Mathlib integer induction principles, at the pinned mathlib revision.

S9. Placed simplification corpus, adoption record, and incoming index. Exact archive-member references and digests are in the exclusion audit.